HANDBUCH ZUM VORTRAG · STAND 10. NOVEMBER 2026

Vom Token zur Entscheidung

Alle Begriffe und Entscheidungsregeln des Vortrags zum Nachschlagen: wie ein LLM funktioniert, was daraus folgt, und wie man GitHub Copilot in VS Code wirksam und kostenbewusst einsetzt. Fachbegriffe stehen so da, wie sie in Dokumentation und Tools vorkommen: auf Englisch.

1 Grundbegriffe
2 Neural Network und Training
3 Von Text zu Vorhersage
4 Wissen, Context, Hallucination
5 Systeme um das Modell
6 Folgen der Funktionsweise
7 Modellwahl
8 Kosten und AI Credits
9 Deutsch oder Englisch
10 Context Engineering
11 Prompting
12 Copilot anpassen: sieben Bausteine
13 Autonomie und Kontrolle
14 Sicherheit und Überprüfung
15 Zehn Regeln
16 Quellen und Stand

01 Wie ein LLM funktioniert

1 Grundbegriffe

EINORDNUNG DER BEGRIFFE

Artificial Intelligence (AI) — Sammelbegriff für Systeme, die Aufgaben lösen, für die man Intelligenz voraussetzen würde. Der Begriff legt keine bestimmte Technik fest.

Machine Learning (ML) — Die Regeln werden nicht programmiert, sondern von einem Suchverfahren aus Beispielen ermittelt. Ergebnis ist ein **Model**: eine Funktion mit gelernten Zahlen.

Deep Learning — Machine Learning mit Neural Networks aus vielen Layern. Grundlage aller heutigen Sprach- und Bildmodelle.

Generative AI — Modelle, die neue Inhalte erzeugen (Text, Code, Bilder, Audio) statt nur zu klassifizieren.

Large Language Model (LLM) — Ein sehr großes Neural Network in Transformer-Architektur, trainiert darauf, das nächste Token eines Textes vorherzusagen. Beispiele: Claude, GPT, Gemini.

Supervised Learning — Training mit Eingaben und von Menschen vorgegebenen Soll-Ausgaben (Labels).

Self-Supervised Learning — Das Soll wird aus den Daten selbst gebildet: nächstes Token verdecken und vorhersagen. Labels von Menschen sind nicht nötig, daher steht sehr viel Trainingsmaterial zur Verfügung. So werden LLMs trainiert.

Parameter — Die gelernten Zahlen eines Modells: **Weights** und **Biases**. Ein aktuelles großes LLM hat größenordnungsmäßig 10^{11} davon.

Hyperparameter — Einstellungen, die ein Mensch vor dem Training festlegt und die nicht gelernt werden: Anzahl der Layer, Learning Rate, Batch Size.

2 Neural Network und Training

WIE DIE PARAMETER ENTSTEHEN

Neuron — Die kleinste Einheit: bildet eine gewichtete Summe seiner Eingaben, addiert einen Bias und wendet eine Activation Function an.

$$z = w_1x_1 + w_2x_2 + b \qquad \hat{y} = \sigma(z)$$

Weight — Faktor, mit dem eine Eingabe multipliziert wird. Bestimmt, wie stark sie zählt.

Bias — Konstante, die zur Summe addiert wird. Verschiebt die Schwelle des Neurons.

Activation Function — Nichtlineare Funktion am Ausgang des Neurons (z. B. Sigmoid, ReLU). Ohne sie ließe sich jedes Netz, unabhängig von der Zahl der Layer, zu einem einzigen linearen Layer zusammenfassen.

Layer — Eine Schicht von Neuronen. Frühe Layer erkennen einfache Muster, spätere kombinieren sie.

Forward Pass — Eine Eingabe läuft einmal durch alle Layer; am Ende steht die Vorhersage.

Loss — Eine einzige Zahl, die sagt, wie falsch die Vorhersage war. Sie ist die vollständige Spezifikation des Trainingsziels: Was der Loss nicht misst, wird nicht gezielt gelernt.

Cross-Entropy Loss — Der Loss für LLMs: −log der Wahrscheinlichkeit, die das Modell dem tatsächlich folgenden Token gegeben hat. Der Loss ist hoch, wenn das Modell dem richtigen Token eine sehr geringe Wahrscheinlichkeit gegeben hat.

Gradient — Die Steigung des Loss bezüglich eines Parameters: Er gibt an, wie sich der Fehler ändert, wenn dieser Parameter leicht erhöht wird.

Gradient Descent — Das Lernverfahren: Jeder Parameter wird ein kleines Stück gegen seinen Gradienten verändert, sodass der Loss sinkt. Der Schritt wird milliardenfach wiederholt.

$$w \leftarrow w - \eta \cdot \partial L/\partial w$$

Learning Rate (η) — Die Schrittweite. Ist sie zu klein, kommt das Training nur langsam voran. Ist sie zu groß, springt jeder Schritt über das Minimum, und das Training divergiert.

Batch / Stochastic Gradient Descent — Der Gradient wird nicht aus allen Daten berechnet, sondern aus einer kleinen zufälligen Stichprobe (Batch) geschätzt.

Backpropagation — Der Algorithmus, der in einem einzigen Rückwärtsdurchlauf (Backward Pass) per Chain Rule die Gradienten **aller** Parameter berechnet. Gradient Descent verwendet diese Gradienten anschließend, um die Parameter anzupassen.

Overfitting / Generalization — Ein Modell kann Trainingsdaten auswendig lernen und bei Neuem versagen. Entscheidend ist das Verhalten auf ungesehenen Daten (Test Set).

Pretraining — Die lange, teure Phase: Next-Token Prediction auf riesigen Textmengen. In dieser Phase entsteht das Wissen des Modells.

Fine-Tuning — Nachtrainieren auf kleineren, gezielten Daten. Bei Chat-Modellen: das Frage-Antwort-Format.

RLHF (Reinforcement Learning from Human Feedback) — Nachtrainieren anhand menschlicher Bewertungen. Prägt Ton, Hilfsbereitschaft und Ablehnungsverhalten, aber auch eine Neigung zuzustimmen.

Training Cutoff — Der Stichtag, an dem die Trainingsdaten enden. Alles danach kennt das Modell nur, wenn es im Context steht.

Inference — Die Nutzung des fertig trainierten Modells. Die Parameter sind eingefroren: Eingaben verändern das Modell nicht.

> → FOLGE FÜR DIE PRAXIS
> Ob ein Modell eine Aufgabe beherrscht, lässt sich nur durch Tests mit bekannten Antworten feststellen.

3 Von Text zu Vorhersage

WAS BEI JEDEM EINZELNEN TOKEN PASSIERT

Token — Ein Textstück aus einem festen Vocabulary: ein Wort, ein Wortteil, ein Satzzeichen. Das Modell sieht ausschließlich Token-IDs, keine Buchstaben.

Tokenizer / Byte-Pair Encoding (BPE) — Zerlegt Text in Tokens. Das Vocabulary entsteht vor dem Training, indem häufige Zeichenfolgen schrittweise verschmolzen werden. Häufige Wörter sind ein Token, seltene bestehen aus mehreren.

Vocabulary — Die Menge aller Tokens eines Modells, typischerweise 100.000–200.000 Einträge.

Embedding — Der Vektor (einige tausend Zahlen), der ein Token repräsentiert. Training ordnet die Vektoren so an, dass ähnlich verwendete Tokens nah beieinander liegen. Grundlage von Semantic Search und RAG.

Attention — Der Mechanismus, mit dem jedes Token Information von anderen Tokens holt. Jede Position stellt eine Query, bietet einen Key und liefert einen Value; das Ergebnis ist eine gewichtete Mischung.

$$\mathrm{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Aufwand wächst quadratisch mit der Länge.

Transformer — Die Architektur aller aktuellen LLMs: mehrere Dutzend gestapelte Blöcke aus Multi-Head Attention (bewegt Information *zwischen* Positionen) und Feed-Forward Network (verarbeitet Information *an* einer Position).

Logits — Die Rohwerte am Ende des Forward Pass: eine Zahl pro Token im Vocabulary.

Softmax — Wandelt Logits in Wahrscheinlichkeiten um, die sich zu 100 % summieren.

Sampling — Das nächste Token wird aus dieser Verteilung **gezogen**, nicht einfach das wahrscheinlichste genommen. Deshalb liefert dieselbe Anfrage verschiedene Antworten.

Temperature — Teilt die Logits vor dem Softmax. $T < 1$ schärft die Verteilung, $T > 1$ flacht sie ab und gibt Unwahrscheinlichem mehr Gewicht.

Top-p — Beschränkt das Ziehen auf die kleinste Menge von Tokens, deren Wahrscheinlichkeiten zusammen p ergeben. Schneidet den langen Rest ab.

Autoregression — Ein Token pro Forward Pass. Es wird an den Context angehängt, dann läuft alles erneut. Daraus folgt: Output-Tokens sind teuer, frühe Fehler pflanzen sich fort, und erzeugter Text ist von zugeliefertem nicht zu unterscheiden.

Reasoning / Thinking — Das Modell erzeugt zuerst Zwischenschritte, vergleichbar mit einem Notizblatt, und verarbeitet sie per Attention, bevor es antwortet. Hilft bei mehrstufigen Problemen.

Reasoning Effort / Thinking Effort — Einstellung pro Aufgabe, wie viele Thinking Tokens das Modell aufwenden darf. Thinking Tokens werden als Output abgerechnet und sind oft nicht sichtbar.

4 Wissen, Context, Hallucination

HERKUNFT UND GRENZEN DES WISSENS

Zwei Wissensquellen

PARAMETER	CONTEXT
Im Training gelernt, danach eingefroren	In diesem Request mitgeschickt
Endet am Training Cutoff	Aktuell und exakt
Kennt nichts Internes	Begrenzt, kostet Tokens
Unscharf, verdichtet	Nach dem Request weg

Memory gibt es im Modell nicht. Was wie Erinnerung wirkt, ist Text, den der Harness erneut in den Context legt.

Context Window — Die maximale Eingabe in Tokens. Der Platz wird geteilt: System Prompt, Tool-Definitionen, Instructions, Dateien, Verlauf und Prompt teilen sich das Fenster. Der gesamte Inhalt wird bei **jedem** Request neu gesendet.

Lost in the Middle — Je voller das Fenster, desto unzuverlässiger nutzt das Modell den Inhalt, am schlechtesten den in der Mitte. Was im Context steht, wird also nicht zwingend verwendet.

Compaction — Nähert sich der Context der Grenze, ersetzt der Harness den bisherigen Verlauf durch eine Zusammenfassung. Details gehen dabei verloren. In der Copilot CLI startet das bei rund 80 %; die Schwellen variieren je Client.

Hallucination — Flüssige, aber falsche Ausgabe. Kein Schritt des Verfahrens prüft eine Aussage gegen die Wirklichkeit. Es ist derselbe Mechanismus, der richtige Antworten erzeugt, angewendet dort, wo die Trainingsdaten ihn nicht festgelegt haben. Gegenmaßnahmen: die Quelle in den Context geben, Tools rechnen und nachschlagen lassen, Belege verlangen und prüfen.

5 Systeme um das Modell

HARNESS, TOOLS UND AGENTS

Harness — Das Programm um das Modell: stellt den Context zusammen, liefert Tool-Definitionen, führt Tool Calls aus, entscheidet über Abbruch und Compaction. GitHub Copilot in VS Code ist ein Harness, Claude Code ein anderer. Dasselbe Modell verhält sich in verschiedenen Harnesses unterschiedlich.

System Prompt — Die Anweisungen des Harness-Herstellers, die vor allem anderen im Context stehen.

Tool — Eine Fähigkeit außerhalb von Text: Datei lesen, Terminalbefehl, Websuche, API-Aufruf. Beschrieben durch Name, Beschreibung und Parameter-Schema.

Tool Call — Das Modell gibt strukturierten Text aus (Tool-Name plus Argumente). Der Harness prüft die Permission, führt aus und hängt das **Tool Result** an den Context. Das Modell führt nie selbst etwas aus.

Agentic Loop — Beobachten, entscheiden, handeln, Ergebnis lesen, wiederholen, bis die Aufgabe erledigt ist. Zuverlässig wird das vor allem durch Rückmeldungen innerhalb der Schleife (Tests, Build).

Agent — Ein Modell in einem Harness mit Tools und einer Agentic Loop. Kann handeln, nicht nur antworten.

Subagent — Ein Agent, den der Haupt-Agent für eine Teilaufgabe startet. Arbeitet in einem **eigenen Context** und liefert nur das Ergebnis zurück. Der Context des Haupt-Agent bleibt dadurch klein.

RAG (Retrieval-Augmented Generation) — Zu einer Frage werden passende Textstellen gesucht (meist über Embeddings) und in den Context gelegt. Wirksamste einzelne Maßnahme gegen Hallucination. Verbleibende Fehler entstehen meist bei der Suche; daher prüfen, welche Textstellen abgerufen wurden.

MCP (Model Context Protocol) — Offener Standard, über den ein Harness externe Tools und Daten anbindet. Ein **MCP Server** ist ein Paket aus Tools.

Prompt Injection — Text in einem Dokument, einer Webseite oder einem Tool Result, der als Anweisung formuliert ist. Funktioniert, weil Anweisungen und Daten denselben Kanal benutzen.

6 Folgen der Funktionsweise

STÄRKEN UND GRENZEN

MECHANISMUS	STÄRKE	GRENZE
Aus Beispielen gelernt	Muster, Sprache, Stil, Code-Idiome	Fähigkeiten lassen sich nur durch Tests feststellen
Next-Token Prediction	Flüssige, plausible Texte in Sekunden	Plausibel heißt nicht richtig; eine Prüfung findet nicht statt
Tokens statt Buchstaben	Jede Sprache, jedes Format	Zählen, exaktes Rechnen, zeichengenaues Arbeiten
Sampling	Varianten, Kreativität	Nicht deterministisch, nicht exakt reproduzierbar
Feste Parameter	Breites Weltwissen sofort verfügbar	Kein Lernen im Betrieb; nichts Aktuelles, nichts Internes
Begrenzter Context	Arbeitet genau mit dem Inhalt des Context	Inhalte außerhalb des Fensters fehlen; Inhalte in der Mitte werden schlechter genutzt
Harness und Tools	Kann lesen, schreiben, ausführen	Handelt auch auf falscher Grundlage

02 Wie man ein LLM einsetzt

7 Modellwahl

DAS KLEINSTE MODELL, DAS DIE AUFGABE LÖST

KATEGORIE	BEISPIELE	INPUT	CACHED	OUTPUT	GEEIGNET FÜR
Lightweight	Claude Haiku 5.5 GPT-6 Luna	$0.10 $0.10	$0.01 $0.01	$0.50 $0.50	Umformatieren, Boilerplate, Commit Messages, einfache Fragen
Versatile	Claude Sonnet 5.5 GPT-6.1 Terra	$2.00 $1.00	$0.10 $0.05	$10.00 $5.00	Tägliche Arbeit: Programmieren, Agent Mode, Review, Tests, Dokumentation
Powerful	Claude Opus 5.5 GPT-6.1 Sol Claude Fable 5.2	$4.00 $2.00 $8.00	$0.20 $0.10 $0.20	$20.00 $10.00 $40.00	Architektur, Debugging über viele Dateien, lange autonome Aufgaben

US$ je 1 Mio. Tokens, Listenpreise und Kategorien laut GitHub Docs, abgerufen am 10.11.2026. Auswahl; welche Modelle freigeschaltet sind, entscheidet die Administration. Preise ändern sich; vor Entscheidungen die Quelle prüfen (Abschnitt 16).

Vorgehen

1. **Ausgangspunkt: Versatile** oder **Auto** (wählt das Modell je Request; wo angeboten, mit „Optimize for“ Efficiency, Balance oder Intelligence).
2. **Wechsel zu Lightweight**, wenn die Aufgabe mechanisch ist und sich das Ergebnis sofort beurteilen lässt.
3. **Wechsel zu Powerful**, wenn die Aufgabe mehrstufig ist, ein früher Fehler teuer wird oder zwei Versuche gescheitert sind.
4. **Thinking Effort** nur erhöhen, wenn die Aufgabe mehrere Schlussfolgerungen erfordert. Beim Nachschlagen und Umformatieren entstehen nur Mehrkosten.

Benchmarks und eigene Evaluation

Benchmarks eignen sich zur Vorauswahl, nicht als alleinige Entscheidungsgrundlage: Keine Rangliste misst die eigene Aufgabe, Testbedingungen variieren, und veröffentlichte Tests landen in späteren Trainingsdaten.

Eigene Evaluation (ein halber Tag): 5–10 echte Aufgaben mit bekanntem Sollergebnis. Kriterien vorher festlegen. Jede Aufgabe dreimal ausführen, weil ein einzelner Durchlauf wegen des Samplings nicht aussagekräftig ist. Kosten und Dauer ebenfalls notieren.

8 Kosten und AI Credits

ABGERECHNET WIRD IN TOKENS

MERKMAL	WERT
1 AI Credit	0,01 US$
Copilot Enterprise	3.900 Credits pro Nutzer und Monat
Copilot Business	1.900 Credits pro Nutzer und Monat
Gemeinsames Kontingent	Über alle Lizenzen der Abrechnungseinheit; kein Übertrag, neues Kontingent am 1. des Monats (UTC)
Kostenlos	Inline Completions und Next Edit Suggestions
Verbraucht Credits	Chat, Agent, Copilot CLI, Cloud Agent, Spaces
Kontingent aufgebraucht	Kostenpflichtige Weiternutzung (Voreinstellung) oder Sperre bis zum Monatswechsel, je nach Richtlinie
Budgets	Pro Nutzer, Cost Center, Organisation, Enterprise

Drei Token-Arten

Input — alles, was gesendet wird: der gesamte Context, bei jedem Request.

Cached Input — ein unveränderter Anfang des Context, den der Anbieter schon verarbeitet hat. Kostet etwa ein Zehntel bis ein Zwanzigstel. Gleichbleibende Instructions senken also die Kosten; ändert sich der Anfang, wird der Cache ungültig.

Output — alles Erzeugte, inklusive Thinking Tokens. Kostet typischerweise das Fünffache des Inputs.

> **Rechenbeispiel** (Sonnet 5.5, Start-Context 30K, 20 Agent-Schritte, +4K je Schritt, 1K Output je Schritt): 1,55 Mio. Input-Tokens, rund **59 Credits** mit Caching, rund **330** ohne. Bei 40 Schritten: 134 Credits.

Kostentreiber

- Lange Sitzungen: Der Verlauf wird jedes Mal mitgesendet, die Input-Tokens wachsen quadratisch
- Große Dateien und Tool Results im Context
- Agentic Loops mit vielen Schritten
- Hoher Thinking Effort für Routineaufgaben
- Powerful-Modell für einfache Aufgaben
- Wiederholungen wegen unklarer Aufgabenstellung

Gegenmaßnahmen

- Neuer Chat pro Aufgabe
- Einzelne Dateien statt ganzer Ordner anhängen
- Modell passend zur Aufgabe wählen
- Präziser Prompt: ein Durchlauf statt drei
- Subagents für Recherche (eigener Context)
- Kurze, gleichbleibende Instructions (der Cache bleibt gültig)

> → FOLGE FÜR DIE PRAXIS
> Die Modellkosten sind meist der kleinere Posten: Zwanzig Minuten Arbeitszeit für die Prüfung kosten mehr als der Durchlauf. Am günstigsten ist das Vorgehen mit den wenigsten Korrekturschleifen.

9 Deutsch oder Englisch

TOKENBEDARF, QUALITÄT UND EMPFEHLUNG JE VERWENDUNG

Kosten: Das Vocabulary ist auf Englisch und Code optimiert. Deutsch braucht für denselben Inhalt mehr Tokens, als Richtwert das 1,3- bis 1,5-Fache, bei Komposita mehr. Das wirkt auf Kosten und auf den Platz im Context Window. Der Faktor hängt vom Tokenizer ab und sollte für eigene Texte gemessen werden. **Qualität:** Aktuelle große Modelle verstehen und schreiben Deutsch sehr gut. Der Abstand zum Englischen wächst bei kleinen Modellen, bei Fachterminologie und dort, wo Dokumentation, Code und Fehlermeldungen ohnehin englisch sind.

VERWENDUNG	EMPFEHLUNG
Instructions, Skills, Agents, Prompt Files	**Englisch.** Werden bei jedem Request gesendet; Begriffe sind eindeutig; teamübergreifend lesbar.
Code, Commits, Fachbegriffe	**Englisch.** Fachbegriffe nicht übersetzen (Gradient Descent, nicht „Gradientenabstieg“).
Einzelne Anfragen im Chat	Die Sprache, in der man am präzisesten formuliert. Die Genauigkeit ist wichtiger als die Sprache.

Ausgabe	Explizit festlegen: `Answer in German.`

10 Context Engineering

BEWUSST ENTSCHEIDEN, WAS DAS MODELL SIEHT

Was im Context steht (von oben nach unten)

1. System Prompt des Harness
2. Tool-Definitionen: integrierte Tools und jeder MCP Server
3. Custom Instructions und Skill-Beschreibungen
4. Angehängte Dateien, geöffnete Datei, Markierung
5. Verlauf und Tool Results (größter Anteil in Agent-Sitzungen)
6. Prompt: der kleinste Teil

In VS Code zeigt ein kleiner Kreis am Chat-Eingabefeld den Füllstand. In der Copilot CLI: `/context` und `/compact`.

Neuer Chat, wenn …

- die Aufgabe wechselt
- drei Korrekturen erfolglos waren (im Context stehen dann drei falsche Ansätze)
- bereits mehrere Compactions stattgefunden haben
- die Arbeit vom Erkunden zum Umsetzen übergeht

Übergabenotiz: Längere Arbeit am Ende einer Sitzung in fünf Zeilen zusammenfassen lassen (Ziel, Stand, Entscheidungen, offene Punkte, nächster Schritt) und die neue Sitzung damit beginnen.

11 Prompting

BAUSTEINE EINES PROMPTS UND BEWÄHRTE VORGEHENSWEISEN

BAUSTEIN	ZU UNGENAU	GENAU
Ziel	„Reparier den Export“	„Der CSV-Export bricht ab 10.000 Zeilen ab. Ziel: läuft für 1 Mio. Zeilen durch.“
Hintergrund	*fehlt*	„Läuft jede Nacht in der CI-Pipeline mit 2 GB Speicherlimit. Streaming ist erlaubt.“
Randbedingungen	„Mach es sauber“	„Öffentliche API nicht ändern. Keine neue Dependency.“
Ausgabeformat	„Zeig mal“	„Erst Plan in fünf Punkten, dann Diff, dann offene Fragen.“
Acceptance Criteria	*fehlt*	„Bestehende Tests grün, plus ein neuer Test mit 1 Mio. Zeilen.“
Umgang mit Lücken	*fehlt*	„Wenn etwas unklar ist: fragen, nicht raten. Annahmen auflisten.“

Die beiden roten Bausteine fehlen am häufigsten und verändern das Ergebnis am stärksten.

Beispiele zeigen (Few-Shot) — Ein gutes Beispiel für die gewünschte Ausgabe wirkt besser als eine lange Beschreibung.

Begründung angeben — „Keine neue Dependency, weil das Audit sechs Wochen dauert“ wird zuverlässiger auf ähnliche Fälle übertragen als ein Verbot ohne Begründung.

Positiv formulieren — Angeben, was geschehen soll, statt nur, was zu unterlassen ist. Aktuelle Modelle befolgen Anweisungen wörtlich.

Erst Plan, dann Ausführung — Ein Plan kostet wenige Tokens und macht Missverständnisse sichtbar, bevor Code entsteht.

Prüfschleife vorgeben — Tests, Build, Linter, Typecheck: Ein Agent, der sein Ergebnis selbst prüfen kann, arbeitet deutlich zuverlässiger.

Neu beginnen statt nachbessern — Nach zwei Fehlversuchen ist ein neuer Chat mit einem besseren Prompt wirksamer als weitere Korrekturen.

Prompting Guides — Anthropic, OpenAI und Google veröffentlichen eigene, modellspezifische Guides. Es lohnt sich, den Guide für das meistgenutzte Modell zu lesen (Abschnitt 16).

12 Copilot anpassen: sieben Bausteine

DATEIEN IM REPOSITORY: VERSIONIERT, IM REVIEW PRÜFBAR, FÜR DAS GANZE TEAM VERFÜGBAR

BAUSTEIN	DATEI	WIRD GELADEN	WOFÜR

Custom Instructions	`.github/copilot-instructions.md`
	`.github/instructions/*.instructions.md`
	`AGENTS.md`
		automatisch, immer	Konventionen, Projektwissen

Prompt Files	`.github/prompts/*.prompt.md`	manuell, per `/name`	Wiederkehrende Anfrage
Agent Skills	`.github/skills/<name>/SKILL.md` `~/.copilot/skills/<name>/SKILL.md`	automatisch, wenn relevant	Verfahren mit Skripten und Vorlagen
Custom Agents	`.github/agents/<name>.agent.md`	manuell, Agent-Dropdown; oder als Subagent	Rolle mit eigenen Tools
MCP Servers	`mcp.json (z. B. .vscode/mcp.json)`	Tools stehen bereit	Externe Systeme und Daten
Hooks (Preview)	`.github/hooks/*.json`	deterministisch, bei Lifecycle-Ereignissen	Regeln, die immer gelten
Agent Plugins (Preview)	Marketplace	nach Installation	Bündel aus Skills, Agents, Hooks, MCP

Custom Instructions

`.github/copilot-instructions.md`

```
## Project
Order service. Java 21, Spring Boot 3, Gradle.

## Conventions
- Run `./gradlew test` before declaring a task
done.
- Never edit files under `generated/`.
- Public API changes need an ADR in `docs/adr/`.

## Language
- Answer in German. Code, comments, commits in
English.
```

`.github/instructions/sql.instructions.md`

```
---
applyTo: "**/*.sql"
---
Use explicit column lists. No `SELECT *`.
```

- Konkrete Entscheidungen festhalten, keine allgemeinen Stilregeln
- Engster Geltungsbereich, der noch alle Fälle erfasst
- Widersprüche entfernen; es gibt keine verlässliche Rangfolge zwischen Regeln
- Regelmäßig kürzen: Jede Zeile wird bei jedem Request abgerechnet

Agent Skills

`.github/skills/ms-style/SKILL.md`

```
---
name: ms-style
description: Write or review user-facing docs
  following the Microsoft Writing Style Guide.
  Use for README files, how-to guides,
  release notes and UI text.
---
# Rules
- Sentence-style capitalization in headings.
- Second person, active voice, present tense.

Approved terms: references/word-list.md
After writing, run scripts/check_terms.py.
```

Progressive Disclosure: (1) `name` und `description` stehen immer im Context, rund 100 Tokens. (2) Die SKILL.md wird erst geladen, wenn das Modell den Skill als passend einstuft. (3) `references/` und `scripts/` nur bei Bedarf.

Die **description** ist entscheidend: Nur sie liegt dem Modell bei der Auswahl vor. Sie sollte angeben, *was* der Skill tut und *wann* er verwendet werden soll.

Eigenen Skill erstellen in fünf Schritten

1. Die Aufgabe dreimal von Hand im Chat lösen
2. Die gleichbleibenden Teile vom Agent in eine SKILL.md schreiben lassen
3. Exakte Teilschritte in Skripte auslagern: rechnen, validieren, formatieren
4. description präzisieren: wann der Skill verwendet werden soll und wann nicht
5. An drei echten Fällen testen, dann per Pull Request ins Repository aufnehmen

Gute Kandidaten: Styleguides (Google developer documentation style guide, Microsoft Writing Style Guide), Design-Vorlagen (Corporate Design, Folienvorlagen), reproduzierbare Abläufe (Release Notes, Incident Report, Migrations- und Review-Checklisten).

Agent Skills sind ein offener Standard: Derselbe Ordner funktioniert in Copilot, Claude Code und anderen Agents. Copilot liest auch `.claude/skills/` und `.agents/skills/`.

Custom Agents

`.github/agents/reviewer.agent.md`

```
---
name: Reviewer
description: Review code for incorrect behavior
  and missing tests without editing files
tools: ['read', 'search']
handoffs:
  - label: Fix findings
    agent: agent
    prompt: Fix the findings listed above.
---
Review the code supplied in the request.
Report findings with file and line references.
Do not edit files or run terminal commands.
```

tools begrenzt, was der Agent tun *kann*. **handoffs** verketten Rollen (Plan → Implement → Review). Anlegen über **Chat: New Custom Agent**.

Abgrenzung: Ein Skill enthält Wissen und Verfahren, die jeder Agent nutzen kann. Ein Agent ist eine Rolle mit eigenem Auftrag und eingeschränkten Tools.

MCP Servers

`.vscode/mcp.json`

```
{
  "servers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/"
    }
  }
}
```

- Ein MCP Server ist fremder Code mit eigenen Zugangsdaten: Herausgeber prüfen, Version pinnen, Credentials beschränken
- Tool-Beschreibungen lesen: Sie werden Teil des Context
- Jede Tool-Definition belegt bei jedem Request Platz im Context: nur aktivieren, was die Aufgabe erfordert

Hooks

`.github/hooks/guard.json`

```
{
  "version": 1,
  "hooks": {
    "preToolUse": [{
      "type": "command",
      "command": "node .github/hooks/guard.cjs",
      "timeoutSec": 10
    }]
  }
}
```

Ein Hook ist ein Shell-Befehl, der deterministisch bei einem Lifecycle-Ereignis ausgeführt wird. Er erhält das Ereignis als JSON über stdin; ein `preToolUse`-Hook kann einen Tool Call mit `"permissionDecision": "deny"` ablehnen.

Ereignisse: `sessionStart`, `userPromptSubmitted`, `preToolUse`, `postToolUse`, `preCompact`, `subagentStart`, `subagentStop`, `agentStop`, `sessionEnd`. In VS Code noch im Preview-Status; Hooks ersetzen weder eine Sandbox noch die Richtlinien der Organisation.

Prompt Files und Agent Plugins

Prompt File — eine gespeicherte Anfrage mit Platzhaltern unter `.github/prompts/`, aufgerufen per `/name` im Chat. Richtig, wenn dieselbe Anfrage regelmäßig wiederkommt, aber kein mehrstufiges Verfahren ist. **Agent Plugin** — ein installierbares Bündel aus Skills, Custom Agents, Hooks, MCP Servers und Slash Commands, verteilt über Marketplaces. Richtig, wenn ein Setup über Teams hinweg geteilt werden soll. Vor der Installation Herausgeber und Inhalt prüfen: Ein Plugin ist ausführbarer Code.

Auswahl der Stufe: nur so viel Struktur wie nötig

STUFE	AUFWAND	ANGEMESSEN, WENN …
Einzelne Anfrage im Chat	keiner	einmalig, oder das Ziel ist noch offen
Prompt mit sechs Bausteinen	Minuten	das Ergebnis weiterverwendet wird
Prompt File	unter einer Stunde	dieselbe Anfrage wiederkommt
Custom Instructions	eine Stunde plus Pflege	eine Regel für viele Aufgaben gilt
Skill	halber Tag plus Pflege	ein mehrstufiges Verfahren regelmäßig läuft
Custom Agent, MCP, Hooks	Einrichtung und Sicherheitsprüfung	Aktionen ausgeführt werden und Regeln verbindlich gelten müssen
Agent Plugin	Paketierung	das Setup teamübergreifend geteilt werden soll
Klassische Software	oft geringer als erwartet	das Verfahren exakt ist und die Eingaben strukturiert sind

13 Autonomie und Kontrolle

ASK, PLAN, AGENT UND DREI ARTEN VON KONTROLLE

AGENT	GEPRÜFT WIRD	GEEIGNET FÜR
Ask	jede Antwort	Verstehen, Optionen abwägen
Plan	den Plan	Größeres und Unklares
Agent	jede Aktion oder nur das Ergebnis	Umsetzen; mit Einzelfreigabe interaktiv, mit Auto-Approve autonom

Fragen vor der Wahl einer höheren Stufe: Ist die Aufgabe eindeutig? Sind die Aktionen umkehrbar? Was kostet ein Fehler? Lässt sich Erfolg automatisch prüfen? Worauf haben die Tools Zugriff?

Mit der geringsten Autonomie beginnen, mit der sich die Aufgabe erledigen lässt. Den ersten Durchlauf immer interaktiv ausführen, auch nach einem Modellwechsel.

ART DER KONTROLLE	BESTIMMT, WAS …
Anweisung Instructions, Prompts, Skills	**wahrscheinlich** passiert
Orchestrierung Agents, Handoffs, Planungsschritt	**üblicherweise** passiert
Technische Beschränkung Tool Approval, Hooks, Permissions, Sandbox	**möglich** ist

> „Ändere nichts unter `prod/` “ in einer Instruction kann das Modell übergehen. Ein fehlendes Schreibrecht oder einen `preToolUse`-Hook mit `deny` kann es nicht umgehen. Teure und nicht umkehrbare Aktionen sind daher über die dritte Art abzusichern.

14 Sicherheit und Überprüfung

PROMPT INJECTION, VERTRAULICHE DATEN, PRÜFUNG DER ERGEBNISSE

Prompt Injection

Der Context besteht nur aus Text. Das Modell kann nicht sicher unterscheiden, ob ein Satz vom Nutzer stammt oder aus einem README, Issue, PDF oder Tool Result. Das ist eine Eigenschaft der Architektur; eine allgemeine Lösung gibt es derzeit nicht.

> **Riskante Kombination:** vertrauliche Daten + nicht vertrauenswürdiger Inhalt + ausgehender Kanal. Treffen alle drei zusammen, kann ein Angreifer Daten abgreifen. Mindestens eines der drei ist auszuschließen.

Gehört nicht in den Prompt: gültige Secrets, Tokens, Passwörter; personenbezogene Daten identifizierbarer Personen; Vertrauliches von Dritten. Gelangt ein Secret in einen Chat, muss es ersetzt werden.

Tool Approvals nicht pauschal auf „Always allow“ stellen, vor allem nicht für Terminal und Netzwerk.

Überprüfung

Vorher: Acceptance Criteria festlegen, bevor die Arbeit beginnt. **Währenddessen:** dem Agent eine Prüfschleife vorgeben. **Danach:** prüfbare Ergebnisse verlangen: Diff statt Beschreibung, Quelle zu jeder Aussage, Liste der Annahmen.

VERWENDUNG DES ERGEBNISSES	PRÜFTIEFE
Erkunden, Prototyp	Durchsicht
Eigene Meinungsbildung	Stichproben
Pull Request, Kollegen	Jede Änderung gelesen und verstanden
Produktion, Vertrag, extern	Vollständig; Zahlen unabhängig nachgerechnet

Besondere Sorgfalt: alle nicht umkehrbaren Aktionen (Senden, Deployen, Löschen), Schreibzugriff auf Geteiltes, erste Durchläufe nach Modell- oder Konfigurationswechsel.

15 Zehn Regeln

FÜR DIE TÄGLICHE ARBEIT

01 Das Modell sagt Tokens vorher. **Die Prüfung** der Ergebnisse bleibt Aufgabe des Menschen.

02 Das Modell kennt nur, was im **Context** steht.

03 Für jede Aufgabe einen **neuen Chat** beginnen.

04 Das **kleinste Modell** wählen, das die Aufgabe löst.

05 Ziel, Randbedingungen und **Acceptance Criteria** vorab festlegen.

06 Zuerst einen **Plan** erstellen lassen, dann den Agent starten.

07 Dem Agent eine **Prüfschleife** vorgeben.

08 Wiederkehrende Abläufe als **Skill** ablegen.

09 Verbindliche Regeln über **Hooks und Permissions** durchsetzen.

10 Dauerhafte Dateien auf **Englisch**, kurz und widerspruchsfrei.

16 Quellen und Stand

PRODUKTBEZOGENE ANGABEN ÄNDERN SICH; MASSGEBLICH SIND DIESE QUELLEN

Offizielle Dokumentation (abgerufen am 10.11.2026)

- GitHub Docs: *Usage-based billing for organizations and enterprises*
- GitHub Docs: *Models and pricing for GitHub Copilot*
- GitHub Docs: *Supported AI models in Copilot*
- GitHub Docs: *Customization cheat sheet*
- GitHub Docs: *Context management*
- VS Code Docs: *Customization*: Custom Agents, Agent Skills, Hooks, Agent Plugins, Language Models
- modelcontextprotocol.io · agentskills.io

Zum Vertiefen

- **3Blue1Brown**, Neural Networks und Transformers: visuelle Erklärung der Mechanismen
- **Andrej Karpathy**, *Deep Dive into LLMs like ChatGPT*
- **Anthropic**, *Building effective agents* und der Prompt Engineering Guide
- **OpenAI** und **Google**: die jeweiligen Prompting Guides
- **Simon Willison**, Sammlung zu Prompt Injection
- **OWASP** Top 10 for LLM Applications

Hinweise zur Genauigkeit. Preise, Credits, Modellnamen und Dateipfade stammen aus den genannten offiziellen Seiten vom 10.11.2026. Der Faktor 1,3–1,5 für deutsche Texte ist ein Richtwert, keine Messung an unseren Texten. Die Zerlegungen der Tokenizer-Demo sowie die Attention-Gewichte und Logits im Vortrag sind illustrativ. Vier Fragen kann nur unsere Administration beantworten: welche Modelle freigeschaltet sind, ob kostenpflichtige Weiternutzung erlaubt ist, welche Budgets gelten und welche MCP Servers und Plugins zugelassen sind.